

Design and Evaluation of a Low-Latency Drone-Based Vision System

Alyaa Alghfeli, Buthaina Almulla, Dilnaz Utemissova, Jenna Khanfar, and Shamma Yaqoob

Mohamed bin Zayed University of Artificial Intelligence (MBZUAI)

Abu Dhabi, United Arab Emirates

{alyaa.alghfeli, buthaina.almulla, dilnaz.utemissova, jenna.khanfar, shamma.yaqoob}@mbzuai.ac.ae

Abstract—This paper presents a controlled empirical study of face recognition under realistic UAV deployment constraints. We combine a live UAV streaming pipeline with a benchmark framework that fixes the detector, preprocessing, gallery construction, and similarity metric while varying only the recognition model. Seven pretrained recognizers, ranging from a 1.1M-parameter SFace model to a 65M-parameter ArcFace-R100 model, are evaluated under a simulated 400 MHz single-core CPU and 128 MB inference-memory budget on the DroneFace dataset, with VGGFace2 used as a generalization probe. We also report per-altitude, per-distance, and per-gender breakdowns. Three findings emerge. First, MobileFaceNet achieves 92.74% accuracy at 16.7 ms latency (59.7 FPS), within approximately three percentage points of the accuracy-best ArcFace-R50 model while running about $39\times$ faster. Second, training-distribution alignment can be as important as architectural capacity: FaceNet performs best on VGGFace2 but degrades substantially on DroneFace, while ArcFace-based models show stronger performance under aerial conditions. Third, CPU-frequency scaling is non-uniform, with mid-weight models benefiting more clearly than very light models dominated by runtime overhead or very heavy models dominated by compute saturation. These results identify MobileFaceNet as the most practical deployment choice under the simulated UAV constraint, and live re-validation on 17 edge-case streaming clips preserves the Phase 2 ranking while exposing a sharp drop in absolute accuracy attributable to threshold-gated matching and to specific lighting and geometry cases.

Index Terms—drone surveillance, face recognition, IoT, real-time inference, edge computing, ArcFace

I. INTRODUCTION

A. Background and Motivation

Unmanned Aerial Vehicles (UAVs) are increasingly used in applications such as public safety, event monitoring, and surveillance, where real-time perception is essential. In these scenarios, drones function as intelligent edge devices that continuously capture and process visual data from dynamic environments. However, achieving reliable face recognition in such settings remains difficult because live UAV operation imposes strict constraints on compute, memory, latency, viewpoint, and image quality.

Many face recognition models perform well on benchmark datasets, but strong offline accuracy does not necessarily translate to strong live deployment performance. In UAV-style streaming, motion blur, compression artifacts, oblique viewpoints, and low-resolution faces can substantially change model behavior. As a result, a recognizer that performs well on

paper may not preserve the same accuracy-efficiency trade-off in a live system.

B. Problem Statement

Face detection and recognition are computationally intensive tasks, particularly when applied to continuous video streams captured by UAVs. While many deep learning models achieve high accuracy under standard conditions, they often assume access to powerful hardware and do not account for the strict computational constraints of embedded systems.

In practice, UAV platforms have limited processing power and memory, and real-time performance is further affected by communication delays and system-level bottlenecks. Different recognition models exhibit very different trade-offs between accuracy, latency, and memory footprint, and the model that performs best on a clean benchmark is not necessarily the model that performs best when placed inside a live streaming pipeline with motion blur and compression artifacts.

However, prior work has not systematically compared lightweight and high-capacity face recognizers under a fixed detector, identical preprocessing and gallery protocol, and UAV-scale CPU and memory constraints, while also considering whether offline benchmark rankings remain meaningful for live streaming deployment.

Therefore, the core problem addressed in this work is how to systematically evaluate and compare face recognition models under realistic UAV deployment constraints, and then validate the selected models against the live streaming conditions of a working drone pipeline, so that the final choice balances accuracy, latency, and computational efficiency.

C. Contributions

This work presents a three-phase study of face recognition for real-time drone surveillance: a live deployment pipeline, a controlled benchmarking framework, and a live re-validation protocol that closes the loop between them¹.

The main contributions of this work are as follows:

- Design and implementation of an end-to-end drone-based vision pipeline in which video capture, face detection,

¹The project code is maintained in two repositories: the benchmarking framework (<https://github.com/jennakhanfar/IOT-DRONE-PROJECT>) and the live drone application (https://github.com/bubulean/drone_project).

embedding-based recognition, and decision logic are executed within a live streaming system, with inference constrained to simulate a low-power onboard UAV compute envelope.

- Development of a controlled benchmarking framework in which the face detection module is kept fixed and multiple lightweight face recognition models are evaluated under identical preprocessing, gallery construction, and similarity metrics, enabling direct and fair comparison.
- Implementation of a hardware-constrained simulation environment that enforces a single-core execution affinity and a hard 128 MB inference-memory budget, reproducing the resource envelope of a typical low-power UAV processor while allowing non-inference pipeline components to run at full host speed.
- Comparative evaluation of seven pretrained face recognition models (SFace, MobileFaceNet, ArcFace, IResNet18/50/100, and FaceNet trained on VGGFace2 and CASIA-WebFace) across two datasets (DroneFace and VGGFace2), including breakdowns by drone altitude, subject distance, and subject gender to surface both performance and fairness trade-offs.
- A live re-validation protocol that feeds the top-ranked benchmark models back into the streaming pipeline to test whether benchmark winners remain winners under real-world motion blur, compression artifacts, and end-to-end latency.

By pairing controlled benchmarking with live re-validation, this work provides practical guidance for selecting and deploying face recognition models in UAV-based surveillance applications.

II. RELATED WORK

A. Real-Time Object Detection for UAV Applications

Real-time object detection is a core component of UAV-based vision systems. The YOLO family of detectors has become widely adopted due to its strong balance between accuracy and inference speed [1]. However, aerial imagery introduces additional challenges, including small object sizes, complex backgrounds, and varying illumination conditions, which can significantly degrade detection performance. Lightweight variants such as Drone-YOLO [2] have been proposed to address these challenges while maintaining real-time throughput on resource-constrained platforms.

Despite these advances, many existing approaches prioritize detection accuracy without explicitly considering strict hardware constraints or deployment feasibility on low-power UAV platforms. In this work, we fix the detection module using a lightweight deep learning detector (UltraFace) and focus instead on evaluating the impact of different recognition models under constrained conditions.

B. Face Recognition Under Aerial Conditions

Deep learning-based face recognition methods, particularly those based on metric learning, have achieved state-of-the-art performance in standard settings. ArcFace [3] introduced an

additive angular margin loss that improves feature discrimination by enforcing tighter intra-class clustering and larger inter-class separation, and this loss has since been adopted by a wide range of backbones, from heavy residual networks (ResNet-50/100, GlintR100) to mobile-oriented architectures such as MobileFaceNet.

However, applying such methods to UAV-captured imagery remains challenging due to low resolution, large camera-to-subject distances, and motion blur [4]. The DroneFace dataset [5] was specifically designed to evaluate recognition performance under these aerial conditions, and prior work has shown that standard off-the-shelf models often struggle to generalize effectively.

Prior drone-based face-recognition work has primarily focused on end-to-end system performance rather than controlled benchmarking of recognition models. Unlike Rostami et al. [4], which evaluate a single pipeline, this work isolates the recognition component by fixing the detection and preprocessing stages and varying only the recognizer. This enables fair comparison across models while explicitly measuring accuracy, latency, memory usage, and deployment trade-offs under simulated UAV hardware constraints.

Modern face recognition systems mainly rely on metric learning frameworks such as FaceNet [8], CosFace [10], and ArcFace [3], which differ in how they enforce inter-class separation in the embedding space. These methods are often implemented within open-source face recognition frameworks such as InsightFace, which provides standardized training pipelines and widely used pretrained ArcFace models for research and deployment.

Lightweight architectures such as MobileFaceNet [9] are designed for mobile and edge deployment, but are sensitive to domain shifts in low-resolution or aerial imagery.

C. Edge Computing and Benchmarking for IoT Vision Systems

In IoT-based vision systems, computational constraints play a central role in system design. Offloading computation to edge or cloud devices is a common strategy to overcome onboard limitations [6]. However, this introduces a trade-off between computational efficiency and communication latency, particularly in UAV systems where wireless transmission quality can vary significantly [7].

While prior work has explored system-level trade-offs in edge computing, there is relatively little systematic benchmarking of face recognition models under realistic UAV deployment constraints. In particular, few studies evaluate how the same recognition pipeline behaves when the only variable is the recognizer, and fewer still examine how model performance breaks down by deployment-relevant conditions such as altitude, subject distance, or protected demographic attributes.

To address this gap, our work adopts a benchmarking-oriented approach. We evaluate multiple recognition models within a unified pipeline, keeping the detection stage and

preprocessing fixed while varying only the recognition component. We simulate hardware constraints to analyze how each model behaves under edge-AI conditions, and we complement the benchmark with a live re-validation step that confirms whether offline rankings translate to streaming deployment. This provides practical, actionable guidance for model selection in UAV applications rather than a purely academic comparison. Unlike prior UAV face-recognition systems that primarily report end-to-end pipeline behavior, our study is benchmark-oriented: it isolates the recognizer, holds the surrounding pipeline fixed, and then re-validates the top models under live streaming conditions.

III. SYSTEM MODEL

The proposed system is organized as a three-phase study: a live drone deployment pipeline (Phase 1) that captures, streams, and processes video in real time; a controlled benchmarking harness (Phase 2) that ranks recognition models under simulated UAV constraints; and a live re-validation protocol (Phase 3) that plugs the top-ranked models back into the live pipeline to verify that benchmark rankings hold under real streaming conditions. This section describes the components of each phase.

A. System Overview

The system operates as an end-to-end vision pipeline, in which the drone captures video and feeds it into a live processing pipeline under a simulated UAV compute envelope. Each frame undergoes face detection, feature extraction, identity matching, and decision making before triggering appropriate actions. The architecture is intentionally modular so that the recognition component can be replaced without modifying the surrounding pipeline, which is essential both for benchmarking and for live re-validation.

B. Live Deployment Pipeline (Phase 1)

1) *Hardware constraint simulation*: Phase 1 inherits the same hardware constraint simulation as the benchmark harness; see Section III-C1.

2) *Drone-side inference subprocess*: To emulate real-world deployment on a resource-constrained UAV, the detection and recognition pipeline is executed within a dedicated subprocess that represents the drone-side computation module. This design separates the system into two components: a laptop-based control and visualization module, and a simulated drone-side inference module responsible for running the core computer vision models. Isolating inference in a separate process prevents blocking of the main control loop and allows the worker to be monitored, constrained, or restarted independently of the user-facing application.

3) *Inter-process communication and low-latency strategy*: Communication between the main application and the detection worker is handled using multiprocessing queues:

- **Frame queue**: transmits raw video frames from the main process to the worker.

- **Result queue**: returns detection outputs, including bounding boxes, identity labels, confidence scores, and per-stage timings.

The frame queue is bounded to a depth of one, so that newly captured frames overwrite older frames whenever the worker is busy. This frame-dropping policy ensures the system always operates on the most recent visual information, trading throughput for latency as required for real-time surveillance. It also prevents unbounded memory growth when inference cannot keep up with the incoming stream, maintaining stable operation under constrained resources.

4) *Face detection module*: The system employs the UltraFace deep learning-based face detector, which provides a favorable trade-off between accuracy and computational cost. UltraFace operates on a fixed input resolution of 320×240 and produces bounding box predictions with confidence scores. A confidence threshold is applied to filter low-probability detections, followed by non-maximum suppression (NMS) to eliminate overlapping boxes. The detector is executed through ONNX Runtime with a single-threaded configuration, which yields deterministic behavior and avoids thread contention under the simulated single-core conditions described in Section III-C1. Compared to classical approaches such as Haar cascades, UltraFace is substantially more robust to variations in lighting, pose, and scale that are typical of UAV-captured imagery, while remaining lightweight enough for real-time deployment.

5) *Face recognition and embedding matching*: For each detected face, the region of interest is cropped, resized to 112×112 (or 160×160 for FaceNet-based backbones), and passed through a recognition model to produce a feature embedding. The embedding is L2-normalized and compared against a gallery of known identities using cosine similarity. This embedding-based formulation allows new identities to be added to the system by simply enrolling their images, without any retraining of the recognition model. The specific models evaluated and ranked are described in Section III-C2.

6) *Known-face gallery*: A gallery of known identities is constructed from images stored in a dedicated directory. Each image is processed through the active recognition model to extract a normalized embedding, which is stored together with its label derived from the filename. Embeddings are cached to disk to avoid recomputation across runs. During inference, each query embedding is compared against the gallery using cosine similarity, and the identity with the highest similarity is selected.

7) *Intruder decision logic and action layer*: Identity assignment is thresholded by a cosine similarity cutoff τ (value in Table VII): if the best match exceeds τ , the corresponding identity is assigned; otherwise, the face is labeled *Unknown*. In the live deployment, τ is set to 0.3 as a permissive operating point for the prototype. This choice is motivated by the degraded visual conditions of UAV video, where motion blur, oblique aerial viewpoints, H.264 compression artifacts, low face resolution, and changing illumination can reduce cosine similarity even for true matches. Increasing τ would make

the system more conservative by reducing false acceptances, but it would also increase false rejections and missed true matches. Decreasing τ would improve recall, but at the cost of more incorrect identity assignments. Therefore, τ is treated as a deployment parameter controlling the known/unknown trade-off rather than as a learned model parameter. An intruder is defined as any detection with a non-*Unknown* label.

For the benchmark experiments in Section IV, τ is not tuned separately for each recognizer. The controlled benchmark uses the same detector, preprocessing, gallery construction, and cosine-similarity protocol for all models, and reports nearest-centroid accuracy. This ensures that the reported ranking reflects recognizer performance rather than threshold calibration. For deployment, τ can be further calibrated with a threshold sweep over false acceptance and false rejection rates using environment-specific validation clips.

To suppress repeated triggers for the same person within a short time window, the action layer maintains three independent per-identity cooldowns—an alert cooldown T_{alert} , a snapshot cooldown T_{snap} , and a tracking cooldown T_{track} (values in Table VII)—each implemented as a last-event timestamp keyed by predicted label. T_{alert} governs the minimum gap between successive audible and visual alerts for the same identity; T_{snap} governs the minimum gap before a new snapshot of the same identity is persisted to disk; and T_{track} governs the minimum gap between drone movement commands issued by the optional tracking mode, which also bounds command-loop latency because the underlying flight-controller call is blocking.

When a detection passes the threshold and falls outside its identity’s alert cooldown, the system triggers an audible alert, a visual notification, and an event-log entry. These actions are executed asynchronously so that they do not block the main processing loop. The system also supports the optional tracking mode, in which the drone follows a selected target based on its position within the frame, gated by T_{track} .

C. Benchmark Harness (Phase 2)

The benchmark harness is designed to rank face recognition models under identical conditions. The detection stage, preprocessing, similarity metric, and gallery construction procedure are all held fixed; only the recognition model is varied between runs. This isolates the effect of the recognizer and enables fair comparison across architectures.

1) *Hardware constraint simulation*: To approximate the resource envelope of a low-power UAV processor, the benchmark harness applies hardware constraints during inference. The target specification is a 400 MHz CPU with a 128 MB memory budget and single-core execution. Constraints are applied exclusively during the forward pass of the recognition model, while data loading, image decoding, and visualization continue to run at full host speed. This isolates the inference workload whose compute envelope we aim to simulate. This design keeps the benchmark focused on recognizer cost rather than on unrelated host-side overheads, making cross-model latency comparisons more interpretable.

Three mechanisms are combined:

a) *Single-core affinity*: The Python process is pinned to a single logical core during inference and returned to all cores afterwards. On Windows this is implemented with `SetProcessAffinityMask`; on Linux, with `sched_setaffinity`.

b) *CPU-frequency throttling*: A daemon thread enforces a duty cycle equal to the ratio of the target drone CPU frequency to the auto-detected host CPU frequency reported by `psutil.cpu_freq`. On a 4.7 GHz host, for example, the 400 MHz target yields a duty cycle of approximately 8.5%. Auto-detection of the host CPU allows the same code to run on different team members’ machines without manual tuning. It should be noted that this sleep-based mechanism reliably attenuates sustained CPU-bound workloads, but for single forward passes shorter than the throttle window its effect on measured latency is limited; this is discussed further in Section IV-D.

c) *Hard memory enforcement*: A monitor thread samples the Resident Set Size (RSS) every 20 ms during inference and computes the delta relative to the baseline captured when inference begins. If the delta exceeds 128 MB, the process is terminated with `SIGKILL`, reproducing the failure mode of an actual out-of-memory event on a 128 MB device. This enforcement is hard rather than advisory, so any configuration that would not fit within the drone’s inference memory budget cannot produce results.

2) *Recognition model suite*: Seven publicly available, pre-trained face recognition models are benchmarked, spanning more than an order of magnitude in parameter count. All models produce fixed-length L2-normalized embeddings and are evaluated off-the-shelf without any fine-tuning on the evaluation datasets. The suite details are in Appendix A.

3) *Datasets*: Two datasets are used, each targeting a different aspect of model behavior:

- **DroneFace** [5]: 1,364 face images of 11 subjects captured from drone-style viewpoints at altitudes of 1.5, 3, 4, and 5 m and at distances between 2 and 16 m. Filenames encode subject identity, altitude, and distance, enabling fine-grained accuracy breakdowns by UAV operating conditions. Gender labels for each subject follow the dataset’s original description (7 male, 4 female) and are used for the fairness analysis in Section IV.
- **VGGFace2**: a diverse, identity-rich dataset used to assess generalization of the same models outside the drone distribution. It contains no UAV-specific metadata, so only overall accuracy is reported.

4) *Evaluation protocol*: For each (model, dataset, CPU target) configuration, all images are passed through the pipeline once to extract embeddings. Per-sample inference latency is recorded under the hardware constraints of Section III-C1. Recognition accuracy is then computed using a leave-one-out nearest-centroid rule: for each sample, a class centroid is recomputed over all other samples of the same identity, all centroids are L2-normalized, and the sample is assigned to the class with the highest cosine similarity. This protocol

is gallery-based and broadly aligned with the live pipeline in III-B, but it is not identical: Phase 2 uses leave-one-out nearest-centroid classification, whereas the live system uses thresholded best-match against enrolled gallery entries.

For each run, we report recognition accuracy, mean and 95th-percentile inference latency, frames per second, model size, and the peak inference-time memory delta. For DroneFace, we additionally report accuracy broken down by drone altitude, subject distance, and subject gender.

5) *Experimental axes*: Experiments sweep three axes derived from the project’s deployment questions:

- **Axis 1 – Model comparison**: all seven models on DroneFace at the 400 MHz target, ranking accuracy/latency/memory.
- **Axis 2 – CPU ablation**: all seven models at CPU targets of 250, 400, and 650 MHz, probing how performance scales with available compute.
- **Axis 3 – Dataset comparison**: all seven models on VGGFace2 at the 400 MHz target, testing whether accuracy rankings on DroneFace transfer to a broader identity distribution.

Each experiment is run in a fresh subprocess and its results written as a self-contained JSON file. The sweep orchestrator skips any experiment whose result file already exists, so interrupted sweeps are resumable without re-running completed work.

D. Live Re-Validation Protocol (Phase 3)

The benchmark harness isolates the recognizer but does not capture effects that are specific to live streaming, such as motion blur, H.264 compression artifacts, variable frame arrival rates, and interactions with the detection stage on real-world faces. To test whether the top-ranked benchmark models retain their ranking under these conditions, we define a live re-validation protocol in which the top two models from Phase 2 are plugged, one at a time, into the live pipeline of Section 3.2. The same set of edge-case clips—covering varied camera angles, subject distances, lighting conditions, and demographic attributes such as head coverings and darker skin tones—is recorded and replayed for each candidate recognizer, and per-frame metrics are logged to `metrics_frames_*.csv`. Comparing the resulting streaming metrics to the benchmark rankings provides a direct answer to the question of whether an offline benchmark winner is also a live deployment winner. Phase 3 is intended as a targeted live re-validation step rather than as a new standalone dataset or benchmark.

IV. RESULTS AND EVALUATION

This section reports the outcomes of the benchmark sweep defined in Section III-C and discusses the resulting trade-offs between accuracy, latency, memory, generalization, and fairness. Unless stated otherwise, all results are obtained with the recognizer running inside the constrained inference context described in Section III-C1. Phase 3 live re-validation results are presented at the end of the section.

A. Experimental Setup

Each (model, dataset, CPU target) configuration is executed as a fresh subprocess and produces a self-contained result file. Per-run metrics include recognition accuracy under the leave-one-out nearest-centroid protocol, mean and 95th-percentile per-frame latency, frames per second, on-disk model size, parameter count, and the peak inference-time memory delta. For DroneFace, additional per-condition metrics are reported (altitude, distance, and gender). Implementation parameters used throughout this study are summarized in Table VII.

The simulated drone target is 400 MHz single-core with a 128 MB RSS-delta budget unless otherwise stated.

B. Model Comparison on DroneFace (Axis 1)

Table I reports accuracy, latency, throughput, model size, and memory delta for all seven recognizers on DroneFace under the 400 MHz constraint.

Several observations follow from Table I. The two heaviest ArcFace variants, R50 and R100, achieve the highest accuracy (95.75% and 93.18%), but their per-frame latency under the 400 MHz constraint is prohibitive for real-time use, reducing throughput to between 0.9 and 1.5 FPS. MobileFaceNet, which has fewer than 3% of the parameters of ArcFace-R50, achieves 92.74% accuracy with a mean latency of 16.7 ms and 59.7 FPS, placing it within three accuracy points of the best model while operating roughly $39\times$ faster. For the deployment envelope targeted by this work, MobileFaceNet therefore represents the practical sweet spot between accuracy and responsiveness.

SFace fails on both deployment axes, achieving only 56.96% accuracy on DroneFace *and* running at 9.2 FPS — roughly $6.5\times$ slower than MobileFaceNet despite having fewer parameters. As shown in the CPU ablation (Section IV-D), SFace’s per-call runtime overhead dominates its tiny forward-pass compute, so frequency scaling does not recover the latency. FaceNet and FaceNet-CASIA also land below MobileFaceNet on every axis: their accuracy is lower, their latency is an order of magnitude higher, and their disk footprint is roughly $8\times$ larger. Under the constraints simulated here, none of FaceNet, FaceNet-CASIA, or SFace is competitive with MobileFaceNet, which leaves the meaningful deployment choice as one between MobileFaceNet and the accuracy-optimal heavy ArcFace variants.

All seven models stay well under the 128 MB inference-delta budget; observed deltas range from effectively zero for MobileFaceNet and SFace up to 14 MB for ArcFace-R50. At this model scale the binding constraint is latency, not inference-time memory. Note that the full weights of ArcFace-R100 (248 MB) exceed the drone’s total RAM budget by construction and so cannot be loaded on-device at all; for the other models, the model-on-disk size is a separate feasibility check from the inference-time memory delta.

C. Dataset Comparison (Axis 3)

To test whether the DroneFace ranking transfers to a broader identity distribution, the same seven models are evaluated on

TABLE I
RECOGNITION PERFORMANCE ON DRONEFACE UNDER THE 400 MHz / 128 MB INFERENCE CONSTRAINT. MODELS ARE SORTED BY ACCURACY. THE Δ RAM COLUMN IS THE PEAK INFERENCE MEMORY DELTA, I.E. THE QUANTITY ENFORCED AGAINST THE 128 MB BUDGET.

Model	Acc. (%)	Latency (ms)	P95 (ms)	FPS	Size (MB)	Δ RAM (MB)
ArcFace-R50	95.75	659.2	902.6	1.5	166.3	14.2
ArcFace-R100	93.18	1152.5	1393.3	0.9	248.6	12.5
MobileFaceNet	92.74	16.7	29.7	59.7	13.0	0.2
ArcFace-R18	86.80	55.2	81.9	18.1	91.8	9.2
FaceNet	78.37	261.4	336.2	3.8	106.8	12.6
FaceNet-CASIA	74.85	241.1	328.0	4.1	110.6	13.8
SFace	56.96	109.1	172.5	9.2	36.9	0.2

VGGFace2 under an identical 400 MHz constraint. Table II summarizes the results.

TABLE II
ACCURACY AND LATENCY ON DRONEFACE VS. VGGFACE2 AT 400 MHZ.

Model	DroneFace		VGGFace2	
	Acc. (%)	Lat. (ms)	Acc. (%)	Lat. (ms)
ArcFace-R100	93.18	1152.5	83.68	1101.2
ArcFace-R50	95.75	659.2	80.57	671.2
FaceNet	78.37	261.4	93.33	77.7
FaceNet-CASIA	74.85	241.1	64.03	78.7
MobileFaceNet	92.74	16.7	71.39	13.4
SFace	56.96	109.1	11.52	30.3
ArcFace-R18	86.80	55.2	48.09	58.9

Two contrasting patterns emerge. FaceNet, which is trained on VGGFace2 itself, achieves its highest accuracy (93.33%) on VGGFace2 and degrades substantially on DroneFace (78.37%), illustrating how training-distribution match can dominate raw architectural capacity. The ArcFace variants show the opposite pattern: their DroneFace accuracy exceeds their VGGFace2 accuracy, indicating that their training objective and data produce embeddings that remain discriminative under the distance, resolution, and viewpoint shifts characteristic of drone imagery. MobileFaceNet generalizes only partially (92.74% on DroneFace, 71.39% on VGGFace2), and SFace effectively collapses on VGGFace2 (11.52%), confirming that it is not a reliable general-purpose recognizer despite its favorable latency.

The key practical takeaway is that the recognizer that wins on a drone-specific dataset is not necessarily the recognizer that wins in general, and a model selected purely on its training-distribution score may underperform in operation.

D. CPU Ablation (Axis 2)

To probe sensitivity to available compute, the seven recognizers were re-run at CPU targets of 250, 400, and 650 MHz while holding all other factors constant. Results appear in Table III.

Three regimes are visible in Table III. The light and mid-weight recognizers that perform real convolutional work (ArcFace-R18, FaceNet, FaceNet-CASIA) scale with CPU frequency as expected: going from 250 to 650 MHz reduces per-frame latency by 14–26%, consistent with the $2.6\times$ increase

TABLE III
EFFECT OF CPU TARGET ON ACCURACY AND LATENCY ON DRONEFACE. ACCURACY IS INDEPENDENT OF CPU TARGET (A RECOGNIZER’S OUTPUT DOES NOT CHANGE WITH COMPUTE BUDGET); THE FREQUENCY AXIS AFFECTS ONLY LATENCY.

Model	250 MHz		400 MHz		650 MHz	
	Acc. (%)	Lat. (ms)	Acc. (%)	Lat. (ms)	Acc. (%)	Lat. (ms)
ArcFace-R100	93.18	1092.3	93.18	1152.5	93.18	1126.2
ArcFace-R50	95.75	615.7	95.75	659.2	95.75	624.6
ArcFace-R18	86.80	64.4	86.80	55.2	86.80	55.6
FaceNet	78.37	298.9	78.37	261.4	78.37	221.3
FaceNet-CASIA	74.85	282.8	74.85	241.1	74.85	223.2
MobileFaceNet	92.74	24.3	92.74	16.7	92.74	19.5
SFace	56.96	105.9	56.96	109.1	56.96	103.2

in nominal core speed being attenuated by the cooperative-throttle granularity and by fixed per-call overhead. The two heaviest models (ArcFace-R50, ArcFace-R100) are essentially flat across frequencies: the 250 $\rightarrow$ 650 MHz range moves ArcFace-R100 from 1092 to 1126 ms, well within run-to-run noise for a 65 M-parameter forward pass. At this capacity, the pinned core is already compute-saturated and the throttle has diminishing effect. Finally, the very small recognizers (MobileFaceNet, SFace) are also nearly flat, but for the opposite reason: their per-frame compute is so small that fixed runtime overhead (input preparation, ONNX/PyTorch session dispatch, and throttle-cycle quantization) dominates, leaving little latency for the CPU to scale against.

Two implications follow for UAV deployment. First, the overall model ranking in Section IV-B is robust to the specific CPU frequency chosen in this range: the top model at 400 MHz is still the top model at 250 and 650 MHz. Second, CPU scaling is a useful knob only for models in the mid-weight band; for very light and very heavy recognizers alike, the effective knobs are model compression, architectural choice, and runtime-level dispatch overhead rather than clock speed.

E. Performance Across UAV Conditions

Because DroneFace filenames encode subject altitude and distance, we can decompose accuracy by UAV operating condition. Tables VIII and IX report this breakdown across six recognizers that span the full accuracy range of the sweep.

Three patterns emerge across models. First, accuracy degrades at the highest altitude tested (5.0 m) for every recog-

nizer, but by different amounts: the heavy ArcFace variants lose 7–14 percentage points while the lighter and VGGFace2-trained models lose more, suggesting that capacity helps absorb the resolution loss at altitude. Second, low altitude (1.5 m) is comparatively benign for the strong ArcFace models (R50, R100, MobileFaceNet all stay above 95%) but penalizes the weaker recognizers heavily (SFace drops to 49.85%), indicating that tolerance to oblique pose is capacity-driven rather than universal.

The distance-axis results are more striking. The ArcFace variants and MobileFaceNet all peak in the 6–12 m window at $> 95\%$ accuracy and degrade gracefully at the extremes; ArcFace-R50 is still at 90% at 16 m, and MobileFaceNet tracks the heavy models closely through most of the range. FaceNet collapses outside a narrow 4–10 m band, falling to 40.45% at 16 m — consistent with it being trained on front-facing web images rather than on aerial crops — and SFace remains weak at every distance, actually performing *better* at 16 m (63.64%) than at 2 m (36.36%), which indicates that its failure modes are a mismatch with the DroneFace identity space rather than a distance effect.

Two operating regimes are therefore visible: a *short-range / oblique* regime (2 m, 1.5 m altitude) in which even strong models lose accuracy due to extreme pose, and a *long-range / low-resolution* regime (14–16 m) in which only the heaviest ArcFace variants retain usable performance. MobileFaceNet falls between these extremes and, given its 59.7 FPS throughput at 400 MHz, best matches the operating envelope expected of a UAV overhead recognizer.

F. Fairness: Accuracy by Gender

Table X reports overall and per-gender accuracy on DroneFace at 400 MHz. Subject gender labels follow the DroneFace documentation (seven male, four female), yielding 868 male and 496 female images.

No model exhibits a strong male-favorable bias on this dataset. The largest single gap, 6.0 percentage points on MobileFaceNet, favors female subjects rather than male subjects. Two caveats apply: the dataset contains only four female subjects, so individual subject effects can strongly influence the per-gender number, and gender is only one of several fairness-relevant attributes. Skin-tone and head-covering effects cannot be isolated on DroneFace because the dataset does not annotate them; these dimensions are addressed in the live re-validation protocol of Section IV-G, which records new clips explicitly covering darker-skin and hijab-wearing subjects.

G. Live Re-Validation Results

The Phase 2 accuracy ranking is preserved when the same recognizers are replayed through the live streaming pipeline, but absolute accuracy drops sharply on both. The top two benchmark recognizers from Section IV-B, MobileFaceNet and ArcFace-R50, were replayed through the live pipeline of Section III-B on a set of 17 single-condition edge-case clips covering hijab, glasses, face mask, steep overhead and side angles, low-contrast and near-dark lighting, and harsh

TABLE IV
PHASE 3 OVERALL LIVE PERFORMANCE. ACC: IDENTIFICATION-CORRECT FRAMES OVER TOTAL FRAMES. ACC_{cond}: SAME NUMERATOR, DENOMINATOR RESTRICTED TO FRAMES WITH A FACE DETECTED.

Recognizer	Acc	Acc _{cond}	Lat (ms)	P95 (ms)	FPS
MobileFaceNet	23.5%	34.0%	74.8	164.6	13.4
ArcFace-R50	28.4%	39.4%	307.3	719.0	3.3

TABLE V
PHASE 2 BENCHMARK VS. PHASE 3 LIVE FOR THE RE-VALIDATED RECOGNIZERS. PHASE 2 FIGURES FROM TABLE I AT 400 MHZ.

Recognizer	Acc		FPS	
	Phase 2	Phase 3	Phase 2	Phase 3
MobileFaceNet	92.74%	23.5%	59.7	13.4
ArcFace-R50	95.75%	28.4%	1.5	3.3

side and overhead sun. ArcFace-R50 required raising the simulated inference-RAM cap from 128 MB to 512 MB to load its 166 MB ONNX, and is therefore included as a memory-relaxed reference rather than a deployable candidate. Both recognizers ran under the same 400 MHz single-core constraint as Phase 2.

ArcFace-R50 leads MobileFaceNet by 3.0 points in benchmark accuracy and by 5.4 points in live conditional identification rate (Table V). The two factors driving the absolute accuracy collapse from 92.74% to 23.5% (MobileFaceNet) and from 95.75% to 28.4% (ArcFace-R50) are not just streaming conditions. First, the matching rule itself is harder live: Phase 2 uses leave-one-out nearest-centroid classification with closed-set *argmax*, whereas the live pipeline uses thresholded best-match against individual gallery entries with a cosine cutoff $\tau=0.3$ and an explicit Unknown outcome (Section III-B). Second, the live throughput compresses the Phase 2 gap between the two recognizers from $40\times$ (59.7 vs. 1.5 FPS) to $4.1\times$ (13.4 vs. 3.3 FPS), reflecting that the live pipeline does not apply the Phase 2 CPU-frequency throttle in the same form, so heavier models benefit disproportionately. Thus, the live FPS figures should not be interpreted as contradicting the benchmark latency ranking; they reflect a different systems context rather than a reversal of recognizer efficiency.

The per-clip breakdown in Table XI (Appendix E) separates detection failures from recognizer failures. Two patterns dominate. On the occlusion clips, the upstream UltraFace detector is the bottleneck: `kevin_glasses` drops to 26.8% / 35.8% detection on the two recognizers and `kevin_mask` to 47.2% / 21.7%; yet on the frames where a face was detected, both recognizers identified `kevin_mask` on every frame and `kevin_glasses` on a majority of frames. The geometric and low-light clips show the opposite pattern: detection rates stay above 60% but conditional identification collapses to 0% on six clips for both recognizers. On every zero-id frame the dominant predicted token was Unknown despite an enrolled identity being present, indicating a similarity-below- τ miss rather than confusion with another enrolled identity. Cleanly

handled cases include `mike_sidesun` (100% conditional id on both recognizers), `mike_sunabove` (95.7% / 100%), and `rohan_sunsid` (58.7% / 100%). The catalogue contains no pure-unknown clip, so false-alarm rate cannot be assessed from this set.

The live evidence supports two of the three Phase 2 conclusions directly. MobileFaceNet remains the only deployment-feasible recognizer of the pair — ArcFace-R50 required relaxing the simulated RAM cap to load at all — and runs $4.1\times$ faster than R50 in the live pipeline, consistent with the practical-winner claim. The 3.0-point benchmark accuracy margin and 5.4-point live conditional-id margin are comparable in scale, consistent with the Phase 2 finding that architectural capacity alone does not dominate when training distribution is matched. Live re-validation does not directly retest the third Phase 2 finding on non-uniform CPU-frequency scaling, since the live pipeline does not sweep frequency.

H. Discussion

Taken together, the benchmark and live re-validation results support three main findings.

First, on DroneFace, MobileFaceNet is the strongest practical recognizer. It reaches within three accuracy points of the highest-accuracy model (ArcFace-R50) while running roughly $39\times$ faster under the 400 MHz constraint, and it is the only one of the two re-validated recognizers that fits the on-device RAM budget without relaxation. Phase 3 confirms this trade-off under live streaming conditions: although ArcFace-R50 retains a 5.4-point conditional-id margin over MobileFaceNet (comparable in scale to its 3.0-point benchmark margin), it runs $4.1\times$ slower in the live pipeline and required raising the simulated RAM cap from 128 MB to 512 MB to load at all. Raw model capacity therefore does not dominate when deployment constraints are taken seriously.

Second, training-distribution match is as influential as architecture. FaceNet wins on VGGFace2 and loses on DroneFace, while the ArcFace variants show the opposite pattern. This argues strongly for evaluating candidate models on data drawn from the intended deployment distribution rather than on generic recognition benchmarks alone.

Third, CPU-frequency scaling is not a universal lever. Both very heavy recognizers (compute-saturated at the pinned core) and very light recognizers (dominated by runtime dispatch overhead) show nearly flat latency curves across the 250–650 MHz range. CPU frequency is a useful knob only in a mid-weight band, which reinforces architectural choice as the primary deployment decision.

A secondary observation concerns the binding constraint. Under the 128 MB inference-delta budget simulated here, the limiting factor for the model sizes we tested is latency rather than memory: the heavier models exceed the drone’s total RAM budget by construction and so could not be loaded on-device at all, but their inference activations remain small (≤ 15 MB). A follow-up compression study reinforces this: quantized MobileNetV3-Small fits within the 128 MB budget

at 4.14 MB but achieves only 7.09 FPS under the same constraint, roughly $8\times$ slower than off-the-shelf MobileFaceNet. Compression alone cannot substitute for a purpose-built face recognizer.

Phase 3 also surfaces a failure mode that Phase 2 cannot. Across 17 single-condition edge-case clips, the dominant failure on geometric and low-light cases is a similarity-below- τ miss (predicted `Unknown` despite an enrolled identity being present), not a confusion with another enrolled identity. This suggests that the threshold τ rather than the recognizer is the next deployment lever to tune, and motivates a per-environment threshold sweep before fielding the system.

V. CONCLUSION

This work demonstrates that, under a simulated 400 MHz single-core CPU and 128 MB inference-memory constraint, MobileFaceNet provides the strongest practical trade-off for UAV-style face recognition. It achieves 92.74% accuracy on DroneFace with 16.7 ms latency, placing it within approximately three percentage points of the accuracy-best ArcFace-R50 model while running about $39\times$ faster. The results also show that training-distribution alignment is as influential as model capacity: FaceNet performs strongly on VGGFace2 but degrades on DroneFace, while ArcFace-based models are more robust under aerial face-recognition conditions. The CPU ablation further shows that frequency scaling is only useful for some mid-weight models, whereas very light models are dominated by runtime overhead and very heavy models by compute saturation.

These findings also expose limitations in the current evaluation. The hardware constraint is simulated using an x86 CPU throttle, so the latency results are useful for ranking models but are not cycle-accurate for a real onboard processor. DroneFace is also small, with only 11 identities, so per-gender gaps may reflect individual subject variation rather than systematic bias. In addition, the dataset lacks annotations for skin tone, head coverings, weather, and extreme oblique angles. High accuracy on DroneFace may therefore reflect both genuine model robustness and the relative simplicity of the benchmark.

Future work will focus on creating a more diverse and realistic drone face-recognition benchmark, expanding evaluation beyond gender-based metrics, and expanding live re-validation with more identities, unknown-subject clips, and broader edge cases. We also plan to deploy MobileFaceNet on a real onboard ARM target to compare the simulated 400 MHz / 128 MB results against physical drone constraints. Overall, the contribution of this work is not only the benchmark ranking itself, but also the gap it reveals between offline evaluation, simplified drone datasets, and real streaming deployment.

REFERENCES

- [1] J. Terven, D. M. Córdova-Esparza, and J. A. Romero-González, “A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS,” *Machine Learning and Knowledge Extraction*, vol. 5, no. 4, pp. 1680–1716, 2023.
- [2] Z. Zhang, “Drone-YOLO: An Efficient Neural Network Method for Target Detection in Drone Images,” *Drones*, vol. 7, no. 8, p. 526, 2023.

- [3] J. Deng, J. Guo, N. Xue, and S. Zafeiriou, "ArcFace: Additive Angular Margin Loss for Deep Face Recognition," in *Proc. IEEE/CVF CVPR*, 2019, pp. 4690–4699.
- [4] M. Rostami, A. Farajollahi, and H. Parvin, "Deep Learning-Based Face Detection and Recognition on Drones," *J. Ambient Intelligence and Humanized Computing*, vol. 15, pp. 373–387, 2024.
- [5] H.-J. Hsu and K.-T. Chen, "DroneFace: An Open Dataset for Drone Research," in *Proc. ACM MMSys*, 2017.
- [6] D. Wang, K. Abu Bakar, and B. Isyaku, "A Comprehensive Review on Internet of Things Task Offloading in Multi-Access Edge Computing," *Heliyon*, vol. 10, no. 9, e29916, 2024.
- [7] R. Xu, S. Razavi, and R. Zheng, "Edge Video Analytics: A Survey on Applications, Systems and Enabling Techniques," *IEEE Communications Surveys & Tutorials*, vol. 25, no. 4, pp. 2951–2982, 2023.
- [8] F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A Unified Embedding for Face Recognition and Clustering," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2015.
- [9] S. Chen, Y. Liu, X. Gao, and Z. Han, "MobileFaceNets: Efficient CNNs for Accurate Real-Time Face Verification on Mobile Devices," arXiv preprint arXiv:1804.07573, 2018.
- [10] H. Wang, Y. Wang, Z. Zhou, X. Ji, D. Gong, J. Zhou, Z. Li, and W. Liu, "CosFace: Large Margin Cosine Loss for Deep Face Recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2018.

APPENDIX

A. Recognition Model Suite

This appendix subsection lists the face recognition models evaluated in the benchmark.

TABLE VI
FACE RECOGNITION MODELS EVALUATED IN THE BENCHMARK.

Model	Backbone	Training set	Params
SFace	SFace-small	OpenCV Zoo	1.1 M
MobileFaceNet	MobileFaceNet	WebFace600K	1.2 M
ArcFace-R18	IResNet-18	MS1MV2	12.4 M
FaceNet	InceptionResNetV1	VGGFace2	27.9 M
FaceNet-CASIA	InceptionResNetV1	CASIA-WebFace	28.9 M
ArcFace-R50	ResNet-50	WebFace600K	43.6 M
ArcFace-R100	GlintR100	Glint360K	65.0 M

B. Implementation Parameters

This appendix subsection summarizes the implementation parameters used across all phases.

TABLE VII
FULL IMPLEMENTATION PARAMETERS ACROSS ALL PHASES.

Parameter	Value
<i>Phase 1: live deployment</i>	
Cosine similarity threshold τ	0.3
Alert cooldown T_{alert}	5.0 s
Snapshot cooldown T_{snap}	10.0 s
Tracking cooldown T_{track}	1.5 s
Frame queue depth	1 (drop-old)
Detector input resolution	320×240
<i>Phase 2: benchmark harness</i>	
CPU target frequencies	250 / 400 / 650 MHz
Inference RAM budget (delta)	128 MB
RSS sampling interval	20 ms
Throttle window	50 ms
Recognizer resolution	112×112 (160×160 FaceNet)
<i>Hardware host</i>	
Host CPU	AMD Ryzen 5 7600X
Host RAM	32 GB
Software stack	Python 3.6, PyTorch, ONNX (CPU)

C. Per-Altitude Accuracy on DroneFace

Table VIII reports recognition accuracy by drone altitude.

TABLE VIII
ACCURACY (%) BY DRONE ALTITUDE ON DRONEFACE AT 400 MHZ.

Altitude	R50	R100	MFN	R18	FaceNet	SFace
1.5 m	98.53	97.95	95.60	87.98	78.30	49.85
3.0 m	97.95	97.07	95.31	89.74	80.06	52.49
4.0 m	97.07	94.43	95.60	88.86	81.82	63.64
5.0 m	89.44	83.28	84.46	80.65	73.31	61.88

D. Per-Distance Accuracy on DroneFace

Table IX reports recognition accuracy by subject distance.

TABLE IX
ACCURACY (%) BY SUBJECT DISTANCE ON DRONEFACE AT 400 MHZ.

Distance	R50	R100	MFN	R18	FaceNet	SFace
2 m	79.55	77.27	78.41	62.50	73.86	36.36
4 m	95.45	94.55	95.00	81.36	90.45	40.45
6 m	99.24	99.24	99.24	88.64	96.97	56.06
8 m	100.00	99.09	99.09	95.00	97.27	59.55
10 m	99.24	96.97	96.97	92.42	91.67	62.88
12 m	99.09	97.73	98.18	95.91	78.64	66.36
14 m	96.97	91.67	89.39	87.12	60.61	62.12
16 m	90.00	82.73	80.00	80.00	40.45	63.64

E. Per-Gender Accuracy on DroneFace

Table X summarizes overall and per-gender recognition accuracy.

TABLE X
PER-GENDER ACCURACY ON DRONEFACE AT 400 MHZ. $n_F = 496$, $n_M = 868$.

Model	Overall (%)	Female (%)	Male (%)
ArcFace-R50	95.75	96.98	95.05
ArcFace-R100	93.18	93.15	93.20
MobileFaceNet	92.74	96.57	90.55
ArcFace-R18	86.80	83.67	88.59
FaceNet	78.37	76.81	79.26
FaceNet-CASIA	74.85	73.59	75.58
SFace	56.96	54.03	58.64

F. Phase 3 Per-Clip Results

Table XI reports per-clip detection and identification results for the two re-validated recognizers.

TABLE XI
PHASE 3 PER-CLIP RESULTS. *det*: FRACTION OF FRAMES WITH A FACE DETECTED. *id*: IDENTIFICATION-CORRECT FRACTION AMONG DETECTED FRAMES. WITH ONE CLIP PER CONDITION, RESULTS ARE DESCRIPTIVE.

clip_id	edge case	MobileFaceNet		ArcFace-R50	
		det %	id %	det %	id %
buthaina_2m	hijab	78.5	47.0	76.5	56.7
buthaina_3m	hijab	73.5	24.6	67.6	48.4
kevin_glasses	glasses	26.8	72.7	35.8	38.6
kevin_mask	face mask	47.2	100.0	21.7	100.0
marshall_overhead40	overhead 40°	56.9	71.4	34.1	40.5
marshall_overhead4020	overhead 40+20°	71.8	6.7	55.3	42.3
marshall_overhead4040	overhead 40+40°	74.6	0.0	61.2	0.0
marshall_side40	side 40°	68.5	0.8	87.0	0.0
mike_lowcontrst	low contrast	61.6	0.0	82.2	0.0
mike_neardark	near-dark	50.5	0.0	80.4	0.0
mike_sidesun	harsh side sun	70.5	100.0	89.2	100.0
mike_sunabove	harsh overhead sun	73.8	95.7	85.3	100.0
rohan_extremesidelight	extreme side lighting	69.5	18.1	85.4	4.7
rohan_lowcontrst	low contrast	65.8	0.0	89.0	0.0
rohan_neardark	near-dark	64.8	0.0	87.9	0.0
rohan_sunabove	harsh overhead sun	96.7	25.0	84.2	52.5
rohan_sunsides	harsh side sun	93.3	58.7	77.0	100.0